
The Art Gallery Problem

CSE 462: Algorithm Engineering Sessional
Final Presentation

Team Members

Turjoy Dey	(ID: 2005016)
Rafiqul Islam Rayan	(ID: 2005062)
Tahsan Kabir Siam	(ID: 2005064)
Abdullah Faiyaz	(ID: 2005065)
Shanto Mondol	(ID: 2005073)
Swarup Siddharth Mondol	(ID: 2005100)

Department of Computer Science and Engineering (CSE)
Bangladesh University of Engineering and Technology (BUET)

April 2026

Presentation Outline

• Part 1: Foundations

① Problem Definition

- Formal definition & mathematical formulation
- Complexity class & hardness analysis

② Existing Algorithms Survey

- Chronological overview of approaches
- Academic references & brief descriptions

③ Chosen Algorithms in Detail

- Tozoni et al. ILP-based iterative solver
- Hengeveld & Miltzow vision-stable solver

• Part 2: Contributions

④ Proposed Modifications

- AABB pre-filter & GLPK integration

⑤ Experimental Setup

- AGPLib benchmark datasets
- Comparison protocol

⑥ Experimental Results

- Cross-solver comparison
- Modification impact analysis

Section 1

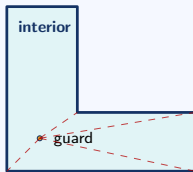
Problem Definition

Formal definition & mathematical formulation

Complexity class & hardness analysis

The Art Gallery Security Challenge

Securing a Modern Art Gallery



Dashed: first segments of sight from a guard toward boundary vertices (schematic).

The floor plan is **not** a rectangle: it has corners, alcoves, and corridors. In applications (museums, floor monitoring, robotics), we abstract this layout as a **simple polygon** in the plane: walls are edges, free space is the interior. The diagram shows one orthogonal example; real galleries can be even more jagged.

The Real-World Problem

- **Irregularly shaped** gallery space — no single “obvious” place to stand
- Security **guards** (or cameras) are **expensive** to hire, power, or maintain
- We want to **minimize** their number while ensuring **every** location on the floor is observed

What “covering” means informally

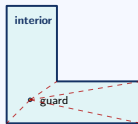
A point on the floor is **covered** if some guard has an unobstructed straight line of sight to it. Walls are opaque: there is no seeing through corners or through exterior space.

Formalizing the Security Problem

From Gallery to Geometry

Real World $\rightarrow$ Mathematics

- **Gallery walls** $\rightarrow$ the closed polygonal chain bounding region P
- **Security guard** $\rightarrow$ a point $g \in P$ (we use **point guards** unless stated otherwise)
- **Artwork anywhere** $\rightarrow$ any point $x \in P$ must be “seen” by some guard
- **Line of sight** $\rightarrow$ the Euclidean segment $\overline{gx}$; it must lie entirely in P



Dashed: first segments of sight from a guard toward boundary vertices (schematic).

Orthogonal simple polygon (TikZ schematic, cf. `original_slide.tex`).

Crucial constraints (modeling assumptions)

- **Opaque walls:** if the segment $\overline{gx}$ leaves P or touches the exterior, x is **not** visible from g
- **360° vision:** from a fixed guard position, visibility is isotropic in angle (no “cone” restriction unless we add it later)

Figure (right): interior shaded; ∂P is the thick outline. Dashed segments from the guard illustrate straight-line visibility that must stay inside P .

The Fundamental Question (sufficiency flavour)

“What is the smallest number of guards that **definitely** suffices for **every** layout with n walls?” — Chvátal: at most $\lfloor n/3 \rfloor$. (Finding the minimum for **one** fixed P is the optimality question.)

Formalizing the Problem

* Mathematical Model

Planar embedding: gallery = closed polygonal region; n counts vertices (wall corners).

Given

- A **simple polygon** $P \subset \mathbb{R}^2$: boundary is one closed chain, **no self-crossings**
- Here: **no holes** (simply connected interior). Later experiments also use polygons **with holes** (multiply connected interiors)

Goal (coverage / total visibility)

Choose a finite set $G \subset P$ of guard positions so that **every** point of the gallery is visible from at least one guard:

$$\forall x \in P, \exists g \in G : \text{segment}(x, g) \subseteq P$$

Minimum AGP: smallest $|G|$. Chvátal (next) bounds how large that minimum can be in terms of n alone.

Visibility rule (standard AGP)

Points $p, q \in P$ **see each other** iff the closed segment $\overline{pq}$ is a subset of P . Equivalently, the segment does not cross the exterior and does not “shortcut” through walls.

The $\lfloor n/3 \rfloor$ Security Guarantee

Chvátal's Theorem (1975) [1]

This is a theorem about **sufficiency**: for any simple polygon with n vertices, there *exists* a guard set of size at most $\lfloor n/3 \rfloor$ that covers the whole interior. It does **not** tell you how to find the set in practice, and it does **not** claim that every polygon *needs* that many — only that you never need *more*.

The Promise

“No gallery with n walls needs more than $\lfloor n/3 \rfloor$ guards.”

Small examples (plug in n):

- Triangle ($n = 3$): $\lfloor 3/3 \rfloor = 1$ guard always enough
- Square ($n = 4$): $\lfloor 4/3 \rfloor = 1$ guard
- Hexagon ($n = 6$): $\lfloor 6/3 \rfloor = 2$ guards **suffice in the worst case over all hexagons**

How to read the bound

- $\lfloor n/3 \rfloor$ is a **worst-case upper** bound on how many guards you might need for **one** polygon with n vertices
- Many specific floor plans admit coverage with **fewer** guards than $\lfloor n/3 \rfloor$
- The factor $1/3$ is **sharp** for this style of theorem: you cannot replace $\lfloor n/3 \rfloor$ by a smaller

The $\lfloor n/3 \rfloor$ Security Guarantee (Continued)

Important Distinction

This tells us: “ $\lfloor n/3 \rfloor$ guards are **enough**.”

NOT: “We need exactly $\lfloor n/3 \rfloor$ guards.”

TikZ below = original_slide.tex **hexagon figure**. $n = 6$ pedagogical. **Left:** one center guard covers the convex hexagon (1 suffices). **Right:** two guards with split visibility ($\lfloor 6/3 \rfloor = 2$ may be necessary for some hexagons).



1 guard



$\lfloor 6/3 \rfloor = 2$ guards

Key insight:

$\lfloor n/3 \rfloor = 2$ is the **upper bound**

But some hexagons can be guarded with **only 1 guard**

Key Takeaways

- $\lfloor n/3 \rfloor$ is an **upper bound** on need, not a prescription
- Many galleries need **fewer** guards than the worst-case formula
- The theorem is about **existence** of a small covering set, not uniqueness

Practical implication

For any gallery with n walls:

Security can always be achieved with at most $\lfloor n/3 \rfloor$ guards.

Two Views of the Art Gallery Problem

These two questions use the **same** geometric model (simple polygon, standard visibility) but ask for **different** outputs.

Sufficiency is a *universal* statement: one number $\lfloor n/3 \rfloor$ works for **all** n -vertex polygons. **Optimality** is *instance-specific*: the answer depends on the shape of P and can be much smaller than $\lfloor n/3 \rfloor$.

Sufficiency Question

Given: only the number of vertices n

Question: How many guards are *guaranteed* to be enough for **every** simple gallery with n walls?

Answer: $\lfloor n/3 \rfloor$

Status: **Solved** — Chvátal (1975); constructive proof route via triangulation and 3-coloring (Fisk, 1978 [2])

Interpretation: this is a **classical** combinatorial geometry result. It tells a designer how many guards might be needed in the worst case when only n is known.

Optimality Question

Given: the full **specific** polygon P (coordinates of vertices)

Question: What is the *minimum* number of guards needed to cover **all** of P ?

Answer: **No known polynomial-time exact algorithm** for arbitrary simple polygons in the standard model

Status: NP-hard (vertex guards, Lee & Lin 1986); stronger hardness for point guards — see next slide

Interpretation: this is what **exact solvers** (ILP, vision-stability, etc.) in this project try to approximate or solve: smallest guard set for **one** input floor plan.

Key Takeaway

“The” Art Gallery Problem in algorithm engineering usually refers to the *optimality* question on a **fixed** instance. The sufficiency theorem motivates why $\lfloor n/3 \rfloor$ is a natural **scale** for guard counts, but not the full story for minimum computation.

Hardness Results

- ▶ **NP-hard** for vertex guards in simple polygons (Lee & Lin, 1986 [3])
- ▶ **NP-hard** for point guards (Aggarwal; O'Rourke, 1987 [4])
- ▶ **$\exists\mathbb{R}$ -complete** for point guards (Abrahamsen, Adamaszek & Miltzow, 2018 [5])
- ▶ **APX-hard** for simple polygons (Eidenbenz et al., 2001 [6])

Set Cover Formulation of AGP

AGP as Integer Linear Program (ILP)

Let $G, W \subseteq P$ be finite sets of guard candidates and witnesses. The discretized AGP reduces to a **Set Covering Problem (SCP)**:

$$\text{AGP}(G, W) := \min \sum_{g \in G} x_g \quad \text{s.t.} \quad \sum_{g \in \text{Vis}(w) \cap G} x_g \geq 1, \quad \forall w \in W, \quad x_g \in \{0, 1\}$$

Three AGP Variants

- ▶ $\text{AGP}(P, P)$: point guards, full coverage
- ▶ $\text{AGP}(V, P)$: vertex guards only
- ▶ $\text{AGP}(P, \partial P)$: boundary coverage only

Key Insight

The continuous AGP has **infinite** variables and constraints. Practical algorithms **iteratively discretize** G and W , reducing AGP to a sequence of finite SCP instances solvable by ILP solvers.

Section 2

Existing Algorithms Survey

Chronological overview of algorithmic approaches
with academic references

Approximation & Heuristic Algorithms

Upper-Bound Algorithms: Fast but Non-Optimal

Algorithm	Description	Complexity	Ref
Fisk (1978)	Triangulation + 3-coloring of dual graph; selects smallest color class as guards. Guarantees $\lfloor n/3 \rfloor$.	$O(n)$	[2]
Amit et al. (2007)	Greedy heuristics: 13 strategies using candidate sets and priority functions; factor-2 empirical.	$O(mn)$	[7]
Ghosh (2010)	$O(\log n)$ -approximation for vertex and edge guards via SCP relaxation.	$O(n^2 \log n)$	[8]
Bonnet & Miltzow (2017)	$O(\text{OPT}^2)$ -approximation for point guards; first for general simple polygons.	Poly	[9]

Exact Algorithms for AGP

ILP-Based Iterative Approaches: Provably Optimal Solutions

Algorithm	Description	Scale	Ref
Couto et al. (2011)	Exact vertex-guard solver $\text{AGP}(V, P)$ via iterative SCP; $>10k$ polygons up to 2500 vertices.	≤ 2500	[10]
Kröller et al. (2012)	Primal-dual LP/ILP with cutting planes for $\text{AGP}(P, P)$; fractional & integer point guards.	≤ 1000	[11]
Bottino & Laurentini (2011)	Edge-covering heuristic lifted to interior coverage; near-optimal for small polygons.	≤ 60	[12]
Tozoni et al. (2016)	Iterative ILP; alternates AGPW (lower bound) and AGPFC (upper bound) until gap closes.	≤ 2500	[13]
Hengeveld & Miltzow (2021)	Vision-stability based arrangement refinement + IP; theoretical guarantees for stable polygons.	≤ 500	[14]

Engineering Art Galleries: State of the Art

Comprehensive survey: de Rezende et al. (2016) [15]

Two independent research lines converged into the current frontier:

Campinas (Brazil)

- Iterative SCP via ILP
- Chwa-Points / Convex-Vertices discretization
- Lagrangian heuristic for SCP
- Up to 2500 vertices, 98% optimality

Braunschweig (Germany)

- Primal-dual LP + cutting planes
- CGAL visibility package
- Lazy-exact kernel for speed
- 5000 vertices with holes

Combined C+BS-2013

Joint Campinas–Braunschweig implementation solves most $AGP(P, P)$ instances to proven optimality for polygons up to 2000–5000 vertices [15].

Section 3

Chosen Algorithms in Detail

Algorithm 1: Tozoni et al. ILP-Based Iterative Solver

Algorithm 2: Hengeveld & Miltzow Vision-Stable Solver

Algorithm 1: Tozoni et al. — Overview

A Practical Iterative Algorithm for AGP Using ILP [13]

Iteratively computes **lower bounds** (via AGPW) and **upper bounds** (via AGPFC) until the gap closes to zero.

Lower Bound: AGPW(D)

- 1 Build arrangement $\text{Arr}(D)$ from visibility polygons of witnesses D
- 2 Identify **light AVPs** (maximal visibility faces)
- 3 Guard candidates $C = V_L(D) \cup V$
- 4 Solve AGPWFC(D, C) via ILP

Upper Bound: AGPFC(C)

- 1 Start with witness set D_f
- 2 Solve AGPWFC(D_f, C) via ILP
- 3 If solution covers $P \Rightarrow$ done
- 4 Else add witnesses in uncovered regions, repeat

Solved 2740 instances; 95%+ optimality rate; up to 2500 vertices

Algorithm 1: Key Concepts

Visibility Arrangement

Given a finite set $S \subset P$, overlay all visibility polygons $\text{Vis}(s)$ for $s \in S$. This creates a planar subdivision into **Atomic Visibility Polygons** (AVPs).

Every point in an AVP is seen by the **same set** of guards.

Light & Shadow AVPs

Light AVP: locally maximal — seen by the most guards.

Shadow AVP: locally minimal — hardest to cover.

Placing one witness per shadow AVP suffices to verify full coverage.

Witness Management

Three initialization strategies:

- **Chwa-Points (CP):** midpoints of reflex-reflex edges + convex vertices adjacent to reflex vertices
- **Convex-Vertices (CV):** all convex vertices
- **All-Vertices (AV):** all polygon vertices

CP empirically achieves the best performance.

SCP Acceleration

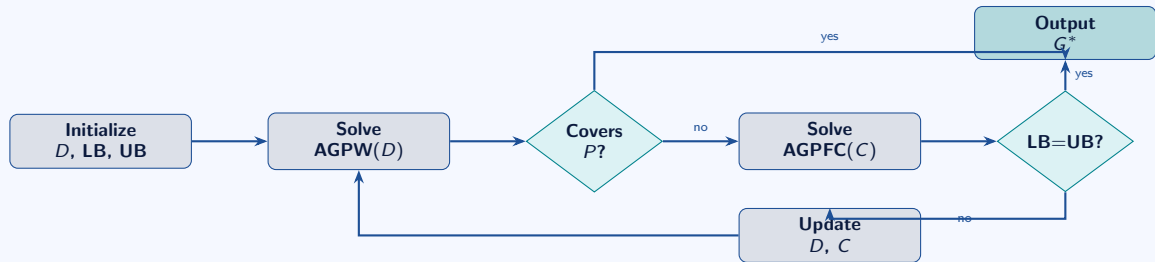
Matrix reduction (redundant columns/rows) +
Lagrangian heuristic for warm-starting the ILP solver.

Algorithm 1: Pseudocode

Algorithm 1 AGP Algorithm (Tozoni et al. [13])

```
1:  $D \leftarrow$  initial witness set (Chwa-Points)
2:  $LB \leftarrow 1$ ,  $UB \leftarrow n$ ,  $G^* \leftarrow V$ 
3: loop
4:   Solve AGPW( $D$ ):  $G_w \leftarrow$  optimal solution,  $z_w \leftarrow |G_w|$ 
5:   if  $G_w$  covers  $P$  then
6:     return  $G_w$ 
7:   end if
8:    $LB \leftarrow \max(LB, z_w)$ 
9:   if  $LB = UB$  then
10:    return  $G^*$ 
11:  end if
12:   $C \leftarrow V_L(D) \cup V \setminus \{\text{light AVP vertices} + \text{polygon vertices}\}$ 
13:   $D_f \leftarrow D \cup$  one point per uncovered region
14:  Solve AGPFC( $C$ ) using  $D_f$ :  $G_f \leftarrow$  optimal,  $z_f \leftarrow |G_f|$ 
15:   $UB \leftarrow \min(UB, z_f)$ ; update  $G^*$  if improved
16:  if  $LB = UB$  then
17:    return  $G_f$ 
18:  end if
19:   $D \leftarrow D \cup$  new witnesses from uncovered regions
20: end loop
```

Algorithm 1: Step-by-Step Flow



Convergence

AGPFC converges in $O((|C| \cdot n)^2)$ iterations (polynomial). In practice: 2–7 iterations for most polygons.

Runtime Split

Preprocessing (visibility, arrangements): 50–85%
ILP solving: 15–50%
Geometric routines dominate, not ILP.

Algorithm 1: Initial Witness Selection (I) — AV & CV

- Tozoni et al. [13], Section 4.4

The iterative solver starts from a finite witness set $D \subset P$. The implementation offers four initialization modes; the first two fix witnesses only at vertices of the polygon boundary (and holes).

1. All-Vertices (AV)

Take **every vertex** of the polygon (outer boundary and holes): $D = V$.

Most conservative: no vertex-visible region is ignored. Cost scales with $|V|$: more witnesses $\Rightarrow$ more visibility polygons, a heavier arrangement $\text{Arr}(D)$, and larger set-cover matrices.

2. Convex-Vertices (CV)

Use only **convex** boundary vertices (interior angle $< 180^\circ$ at the vertex).

Intuition: convex vertices sit at outward “tips” and are among the hardest points to cover; covering them tends to cover much of the interior. This is a **strict subset** of AV (with the analogous rule on holes).

Tozoni et al. report **similar outer-loop iteration counts** to AV but **considerably lower runtime**, because fewer witnesses mean fewer visibility computations, simpler arrangements, and smaller SCP instances.

Algorithm 1: Initial Witness Selection (II) — Chwa-Points (CP)

- Chwa-Points — Chwa et al. (2006)

Chwa et al. showed that certain polygons are **witnessable**: there exists a *finite* witness set such that any guard set covering those witnesses covers all of P . They give a concrete construction for a small such set.

Edge-wise placement along the boundary

Classify each boundary edge by the convexity of its two endpoints (convex vs. reflex in the interior):

- **Convex–reflex** or **reflex–convex**: place a witness at the **convex** endpoint (where the boundary transitions between “easy” and “hard” to see).
- **Reflex–reflex**: place a witness at the **midpoint** of the edge.
- **Convex–convex**: *no* witness (those regions are easy to cover).

The same classification applies to hole boundaries (with orientation taken into account).

Empirical performance (Tozoni et al.)

In their experiments, Chwa-Points was the **best overall** strategy: it solved the most instances to optimality (e.g. 294 of 300 in one suite) and achieved the best average runtime — including large savings vs. All-Vertices on 500-vertex polygons with holes (see their Table I and Figure 9).

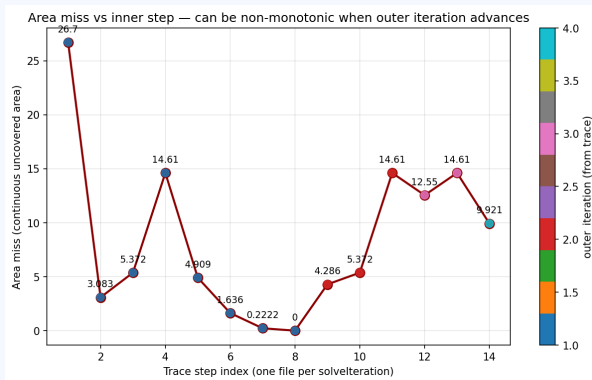
Algorithm 1: Initial Witness Selection (III) — Extended Chwa-Points

4. Chwa-Points Extended

A variant **not** in the paper but available in implementations: same reflex–reflex midpoint rule, but for convex–reflex and reflex–convex edges it places witnesses at **both** endpoints instead of only the convex one.

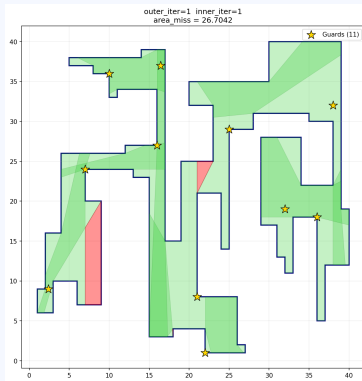
This yields a **superset** of ordinary Chwa-Points witnesses: a slightly larger initial D , in exchange for potentially fewer outer-loop iterations in some instances.

Overview



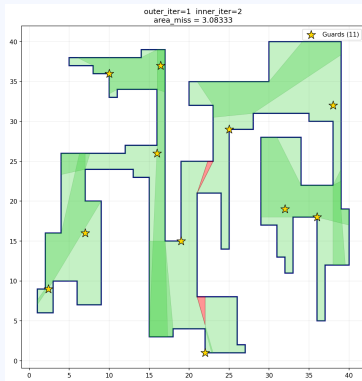
Overview: `area_miss` (uncovered area) vs. inner iteration for this run — the curve shows how the missed fraction evolves as the solver refines guards and witnesses.

Step 1



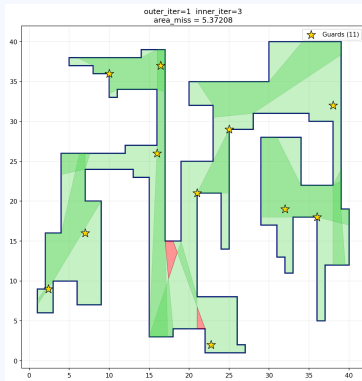
Step 1: Outer Iteration 1 begins: Initial placement of 11 guards leaves a massive red uncovered area.

Step 2



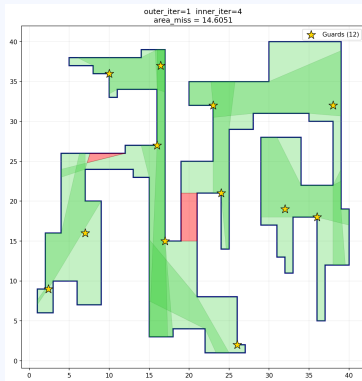
Step 2: The 11 guards shift positions, drastically shrinking the blind spots.

Step 3



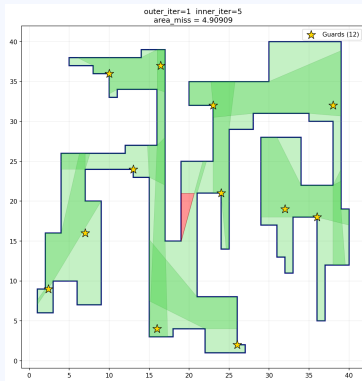
Step 3: A slight adjustment to the 11 guards causes a minor temporary increase in the missed area as it explores.

Step 4



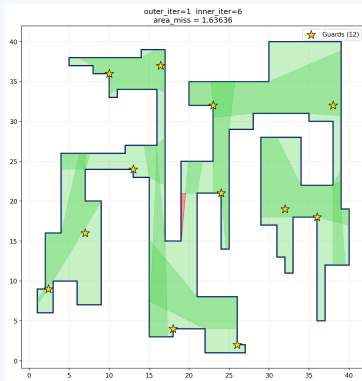
Step 4: The solver jumps to 12 guards (Upper Bound test), initially resulting in new shifted blind spots.

Step 5



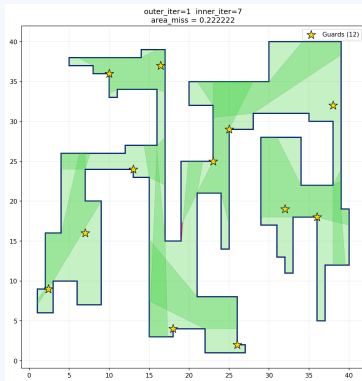
Step 5: The 12 guards are heavily rearranged, significantly reducing the unobserved regions.

Step 6



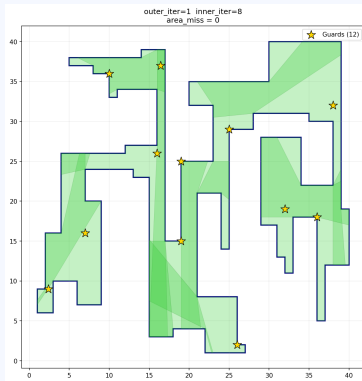
Step 6: Further fine-tuning of the 12 guards minimizes the red blind spots down to tiny slivers.

Step 7



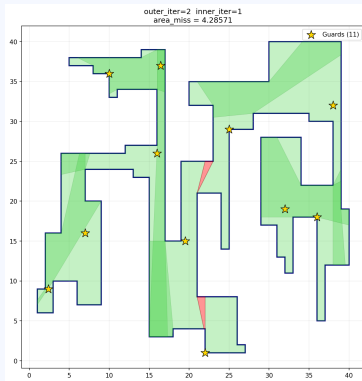
Step 7: The 12-guard layout is nearly perfect, leaving only a fractional amount of uncovered area.

Step 8



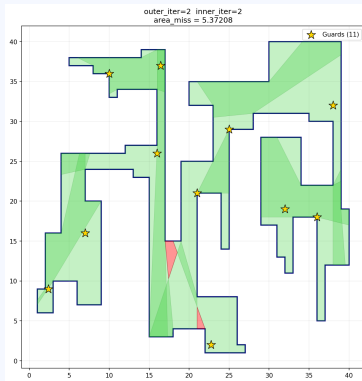
Step 8: Perfect coverage achieved: The 12 guards completely eliminate all blind spots with zero missed area.

Step 9



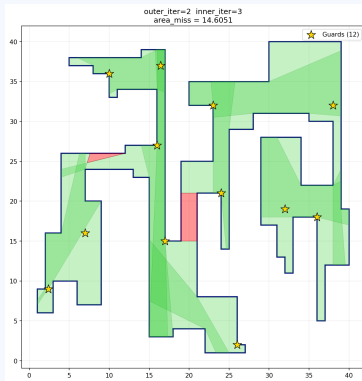
Step 9: Outer Iteration 2 begins: The solver drops back down to 11 guards to test an entirely new layout.

Step 10



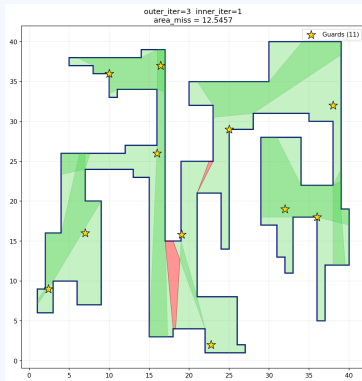
Step 10: The new 11-guard layout is adjusted, but red blind spots stubbornly remain.

Step 11



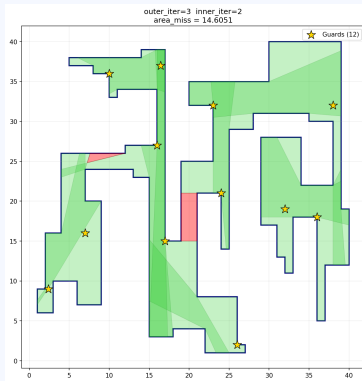
Step 11: Unable to cover the space with 11, the solver reverts to testing a 12-guard configuration.

Step 12



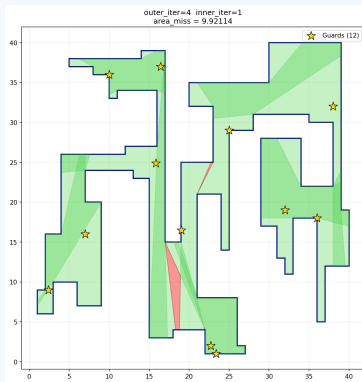
Step 12: Outer Iteration 3 begins: The solver forces yet another unique 11-guard configuration to test for a lower bound.

Step 13



Step 13: This 11-guard attempt also fails, prompting the solver to jump back to 12 guards.

Step 14



Step 14: Outer Iteration 4 begins: The solver runs final checks, proving 11 guards cannot work and 12 is the true minimum.

Algorithm 2: Hengeveld & Miltzow — Overview

A Practical Algorithm with Performance Guarantees [14]

Introduces **vision-stability**: if perturbing guard visibility by $\pm\delta$ doesn't change the optimal guard count, the polygon is **vision-stable**. This enables a provably correct discretization.

Core Idea

Shoot rays from every **reflex vertex** at discrete angles. Intersections within P form the **candidate set** C . Faces of the arrangement form the **witness set** W .

Unique feature: both **vertex-candidates** and **face-candidates** are used as potential guard locations.

Vision-Stability

$\text{vis}_\delta(g)$: enhanced visibility (sees “around corners” by δ)

$\text{vis}_{-\delta}(g)$: diminished visibility

Polygon P is vision-stable δ if:

$$\text{opt}(P, -\delta) = \text{opt}(P, \delta)$$

Most practical polygons are vision-stable.

Algorithm 2: Weak Visibility Polygon Tree

WVPT Construction

- 1 Start with an arbitrary boundary edge e
- 2 Compute weak visibility polygon $\text{vis}(e) \Rightarrow \text{root}$
- 3 For each non-boundary edge e' of $\text{vis}(e)$, recurse into $P \setminus \text{vis}(e)$
- 4 Each node is a weak visibility polygon with a defining chord

Speedup Effect

Two faces can only see each other if their WVPT nodes are siblings or in a parent-child relationship. This **prunes 16–87%** of visibility queries depending on polygon size.

Initial Arrangement

The arrangement $\mathcal{A}$ is constructed by inserting:

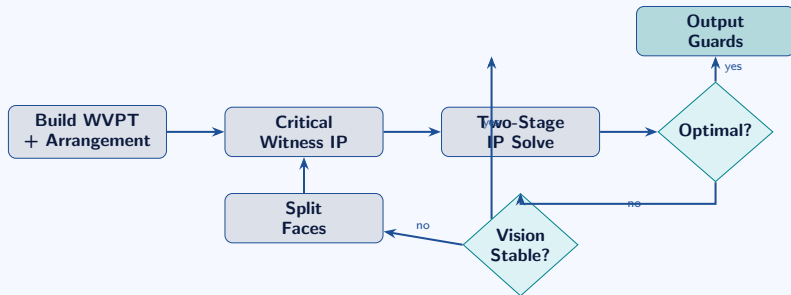
- **Polygon boundary edges**
- **WVPT defining chords**
- **Axis-aligned rays** from reflex vertices (up, down, left, right)

This subdivides P into **faces** (witness regions) and **vertices** (guard candidates).

Local Complexity

Even for 500-vertex polygons, each WVPT node has ~ 28 vertices and ~ 9 reflex vertices on average.

Algorithm 2: Main Iterative Loop



Two-Stage IP

Stage 1: Minimize total guards (vertex + face).

Stage 2: Fix guard count; minimize face-guards and unseen witnesses.

Soft constraint: prefer vertex guards over face guards.

Face Splitting Strategies

Five methods: square split, angular split, reflex chord, extension split, visibility line split.

Creates finer witness regions and new guard candidates.
Granularity λ halves when stuck ($\lambda \rightarrow \lambda/2$).

Algorithm 2: Vision-Stability & Termination

Vision-Stability Check

At each iteration, compute $\delta = 8\pi\lambda$ and solve two modified IPs:

- Enhanced visibility IP ($+\delta$)
- Diminished visibility IP ($-\delta$)

If both return the **same optimal guard count** $\Rightarrow$ vision-stable at this granularity $\Rightarrow$ **provably optimal**.

FPT Results

Corollary: For vision-stable polygons, the one-shot algorithm is FPT with respect to the number of reflex vertices.

Also FPT w.r.t. chord-visibility width.

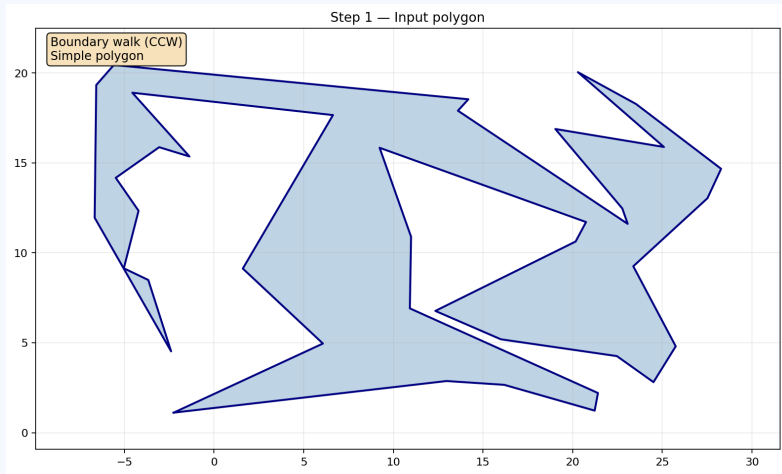
Practical Performance

Comparison with Tozoni et al. on AGPLib [14]:

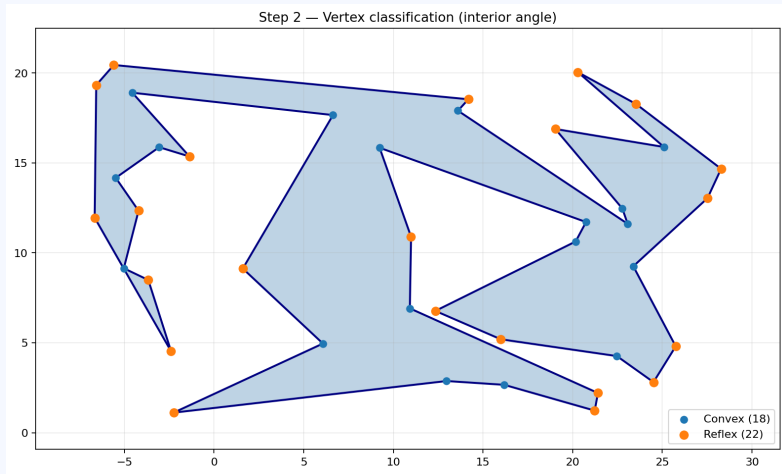
Size	Tozoni (s)	Simon (s)
60	0.18	0.39
100	0.68	0.52
200	2.54	2.02
500	22.34	18.20

Comparable speed; slightly faster on larger instances. IP solving dominates runtime ($>50\%$).

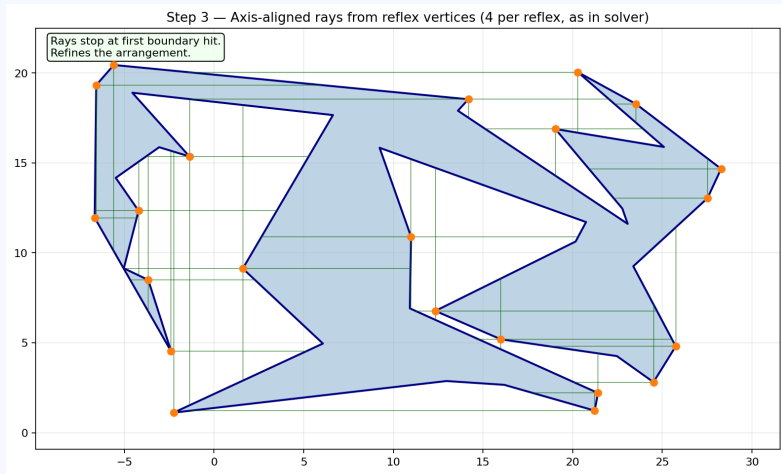
Step 1 — Input polygon



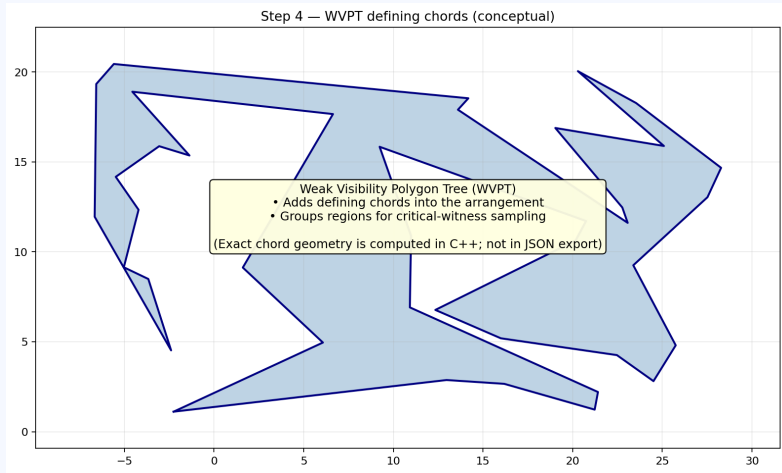
Step 2 — Vertex classification (interior angle)



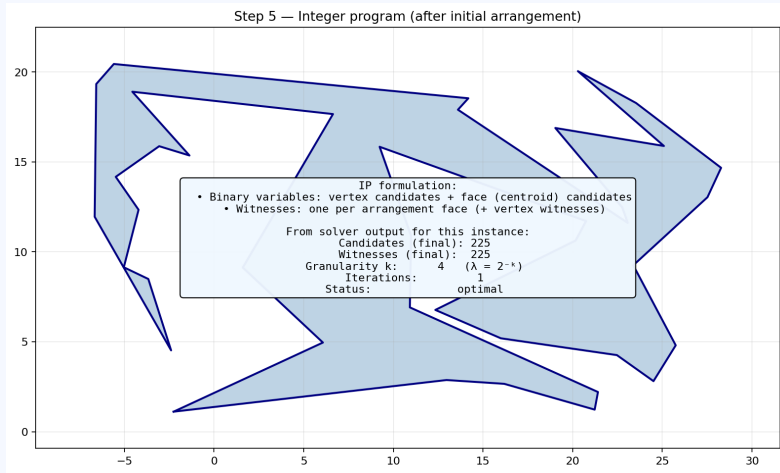
Step 3 — Axis-aligned rays from reflex vertices (4 per reflex, as in solver)



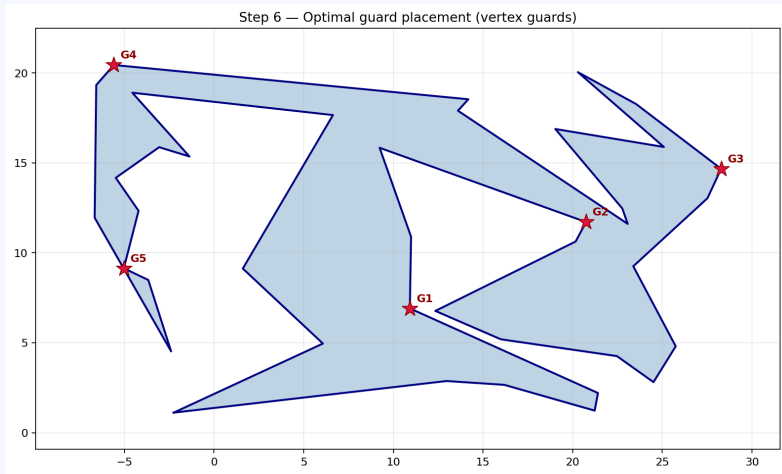
Step 4 — WVPT defining chords (conceptual)



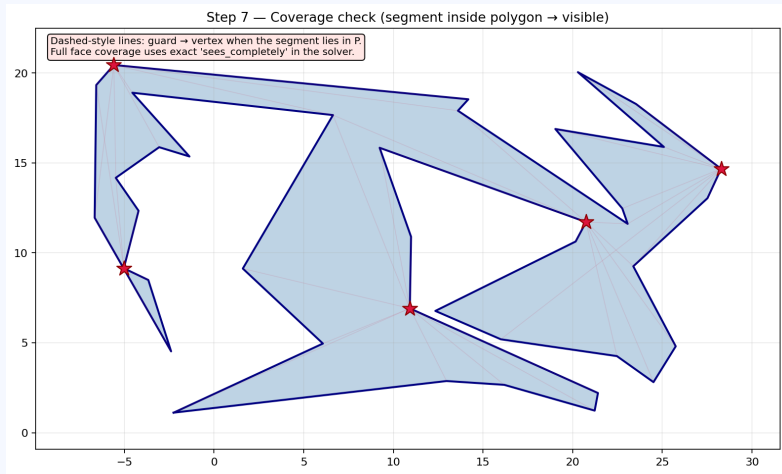
Step 5 — Integer program (after initial arrangement)



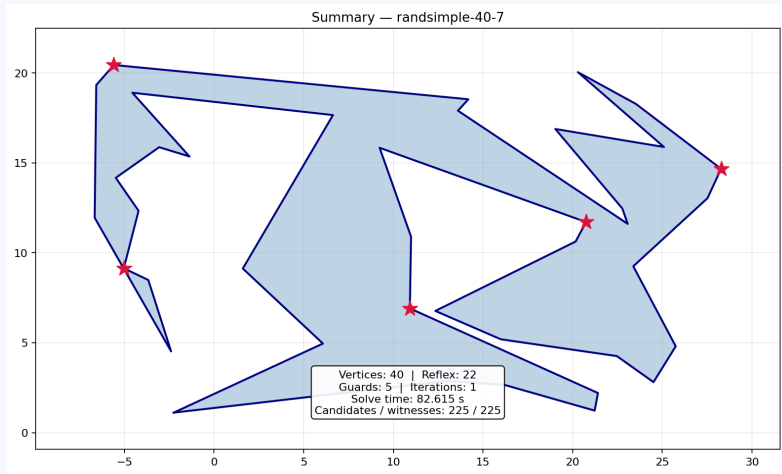
Step 6 — Optimal guard placement (vertex guards)



Step 7 — Coverage check (segment inside polygon $\rightarrow$ visible)



Summary — randsimple-40-7



Section 4

Proposed Modifications

AABB pre-filter (Tozoni) & GLPK integration (reproducibility)

Modification 1: The Visibility-Matrix Bottleneck

Tozoni solver — what `initSolverTime` actually computes

The iterative ILP repeatedly builds a **finite** set cover instance: rows are witnesses $w \in W$, columns are guard candidates $c \in C$. Entry (w, c) is true iff c sees w (equivalently $c \in \text{Vis}(w)$ inside the gallery P).

Why this dominates runtime

Profiling: `initSolverTime` — filling that witness $\times$ candidate matrix — takes about **60–86%** of total time on large instances.

For each pair (w, c) the code must decide whether c lies in w 's **visibility polygon** $\text{Vis}(w)$. The CGAL predicate `bounded_side()` walks the boundary: $O(E)$ work for a polygon with E edges.

Wasted work (geometric intuition)

Naively one pays $O(|W| |C| E)$ -scale cost if every pair runs the expensive test.

Many pairs are impossible: a candidate c can lie far outside the region of the plane that $\text{Vis}(w)$ occupies. Those pairs cannot be visible, yet the baseline code still runs `bounded_side()` **unconditionally**.

Goal

Insert a **cheap necessary condition** before `bounded_side()`: if the condition fails, the candidate is definitely not in $\text{Vis}(w)$, so we skip the $O(E)$ test. The condition must never reject a pair that could be visible — only then is the discretized SCP instance unchanged.

Modification 1: Mode 3 — AABB Pre-Filter & Correctness

Implementation: axis-aligned bounding boxes (AABB)

Original (modes 1 and 2)

- Loops: outer over **candidates** j , inner over **witnesses** i
- Visibility polygon read from `_visCache` and **deep-copied** each time
- `bounded_side()` on every (i, j) pair — no early exit

Mode 3 (new)

- **Loops swapped**: witnesses outer, candidates inner — each `Vis( $w$ )` loaded **once** per row and held by const reference (no copies)
- Precompute an AABB for every candidate and for each witness's `Vis( $w$ )`
- Before `bounded_side()`: `CGAL::do_overlap(candBB[j], visBB)` — $O(1)$
- If boxes are disjoint, $c \notin \text{Vis}(w)$; skip the $O(E)$ edge walk

Correctness (conservative over-approximation)

The axis-aligned box B_w containing `Vis( $w$ )` is a **superset** of `Vis( $w$ )`. If a point lies inside `Vis( $w$ )`, it lies inside B_w ; equivalently, if a candidate lies **outside** B_w , it cannot lie in `Vis( $w$ )`.

The filter may **false-positive** (boxes overlap but $c \notin \text{Vis}(w)$) — we still run `bounded_side()`. It **never false-negates**: we never skip a pair that could be visible. Hence the visibility matrix is **bit-for-bit identical** to the original; guard counts and iterations match.

Also in Mode 3: the same gating applies in `addIterationGridPoints()` during AGPFC iterations (`stepArt`). The pre-solver falls back to **GLPK** + **Lagrangian heuristic** when Xpress is unavailable (see `modification.md`).

Modification 2: GLPK for the Simon (Hengeveld–Miltzow) Solver

Reproducible IP solves without proprietary licenses

Reference stack

The published Hengeveld–Miltzow implementation targets CGAL 4.13+ and uses **IBM CPLEX** 12.10 for the sequence of integer programs (arrangement witnesses, vision-stability checks, face splits).

CPLEX is proprietary and not redistributable, which blocks **out-of-the-box** replication on typical lab machines.

Trade-off: on very large SCP instances GLPK can be slower than CPLEX (cf. Tozoni et al. [13], Table V) — a cost we accept for accessibility.

Our integration

- **GLPK** as the IP backend when CPLEX (or Xpress, where applicable) is unavailable
- Same **Lagrangian** / **warm-start** ideas as in Tozoni Mode 1 for comparable SCP subproblems
- End-to-end pipeline — geometry + IP — runnable with **open-source** tools only

Measured Improvement

initSolverTime: Mode 1 vs. Mode 3

Instance	Candidates	Mode 1	Mode 3	Speedup
randvon-100-24	450	0.097 s	0.058 s	1.7×
randsimple-100-13	416	0.161 s	0.109 s	1.5×
randsimple-80-23	212	0.077 s	0.060 s	1.3×

Scaling Behavior

Improvement scales with:

- Number of guard candidates
- Geometric complexity (more reflex vertices → smaller, localized visibility polygons → higher AABB rejection rate)

Unchanged Outputs

Guard counts and iteration counts are **identical** across modes.

Mode 3 also benefits the `addIterationGridPoints()` function in subsequent AGPFC iterations (same AABB gating).

Section 5

Experimental Setup

AGPLib Benchmark Datasets

Cross-Solver Comparison Protocol

Dataset: AGPLib Benchmark

AGPLib: Standard benchmark for Art Gallery Problem [16]

We use the publicly available AGPLib datasets (Couto et al., 2009; Tozoni et al., 2016), filtering to polygons with ≤ 250 vertices.

Class	Pattern	Sizes Used	Description
agp2009a-orthorand	random- N -id.pol	20–200	Random orthogonal simple
agp2009a-simplerand	randsimple- N -id.pol	20–200	Random non-orthogonal simple
agp2009a-rvk	randvon- N -id.pol	< 300	Random von Koch
gB-simple-simple	gB_simple-simple_*	≤ 200	Simple with holes
gD-ortho-ortho	gD_ortho-ortho_*	≤ 200	Orthogonal with holes

30 instances per size class; 20 size classes; polygon format: .pol

Algorithms Compared

Tozoni (modes 1–3) and Simon on matching benchmark instances

Solver	Description
Tozoni Mode 1	Tozoni et al. iterative ILP with CHWA_POINTS; SCP solved with GLPK + Lagrangian warm-start heuristic .
Tozoni Mode 2	Same framework and witness discretization; SCP solved with plain GLPK (no Lagrangian heuristic).
Tozoni Mode 3	Mode 1 visibility/ILP path + AABB bounding-box pre-filter for the visibility matrix (our modification).
Simon	Hengeveld–Miltzow vision-stable iterative solver with GLPK fallback.

Metrics: guard count, total runtime

Section 6

Experimental Results

Cross-solver comparison & modification impact

Simon vs. Tozoni: Head-to-Head Comparison

Guard count and runtime on matching instances

Instance	V	Simon	Time(s)	Tozoni	Time(s)	Strategy	Δ
random-20-8	20	3	5.111	3	0.006	CHWA_POINTS	0
random-40-14	40	4	36.664	5	0.029	CHWA_POINTS	-1
random-60-2	60	6	128.075	8	0.039	CHWA_POINTS	-2
random-80-23	80	8	305.368	12	0.205	CHWA_POINTS	-4
randsimple-20-24	20	3	7.330	2	0.011	CONVEX_VERTICES	+1
randsimple-40-7	40	5	82.615	5	0.026	CHWA_POINTS	0
randsimple-60-23	60	6	411.972	7	0.069	CONVEX_VERTICES	-1
randsimple-100-13	100	0	0.002	13	0.187	CHWA_POINTS	-13
gB.simple-25:100v-10h_29	25	3	34.704	12	0.887	CHWA_POINTS	-9
Total		38		67			-29

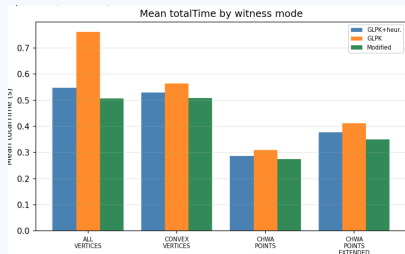
Simon (Hengeveld–Miltzow)

Fewer guards (provably optimal when converged) but **much slower** — seconds to minutes per instance.

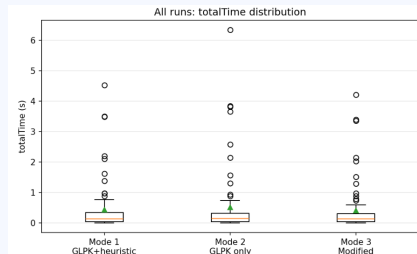
Tozoni

Extremely fast (<1 s for most) but uses more guards as a heuristic/approximate solution (GLPK mode).

AABB Pre-Filter Impact: Mode 1 vs. Mode 3



Mean totalTime by witness strategy: GLPK+heur., GLPK, Modified
(Mode 3)



Distribution of totalTime: Mode 1 (GLPK+heur.) vs. Mode 2 (GLPK)
vs. Mode 3 (Modified)

Key Findings

- AABB pre-filter (Mode 3 / Modified) achieves **1.3–1.7× speedup** on `initSolverTime` vs. Mode 1 (see `modification.md`)
- CHWA_POINTS witness strategy yields the lowest mean totalTime
- Guard counts and iteration counts remain **identical** across modes for the same witness set
- Modified (Mode 3) remains fastest on average in the totalTime plots

Guard Count & Runtime: Cross-Solver Analysis

Comparing guard quality and speed across matching instances

Guard Count Analysis

From the comparison table (9 matching instances):

- Simon total: **38 guards** (proven optimal)
- Tozoni total: **67 guards** (−29 delta)
- Simon finds fewer guards in **7/9** instances
- Gap widens with polygon complexity

Runtime Analysis

- Tozoni: **milliseconds** per instance (<1 s)
- Simon: **seconds to minutes** (5–412 s)
- Simon is $\sim$ **500–6000**× **slower**
- Trade-off: optimality vs. speed

Practical Recommendation

Small polygons (≤ 60 vertices): Both solvers are viable; Simon provides guaranteed optimality.

Large polygons (> 100 vertices): Tozoni is practical; Simon may time out on complex instances.

Reproducibility: GitHub Repository

Single Master Script

All experiments are reproducible with a single command:

```
./run_all.sh    or    python run_all.py
```

Compiles solvers → runs on AGPLib instances → produces comparison plots.

Repository Structure

```
CSE-462-Art-Gallery-Problem/  
  implementation1/      # Tozoni solver (C++, CGAL, GLPK)  
    art-gallery-pg/    # Solver source with Mode 3  
  implementation2/      # Simon solver (C++, CGAL, GLPK)  
  datasets/             # AGPLib .pol files (<250 vertices)  
  results/              # Output logs and plots  
  run_all.sh            # Master script  
  README.md            # Build & run instructions
```

GitHub Repository:

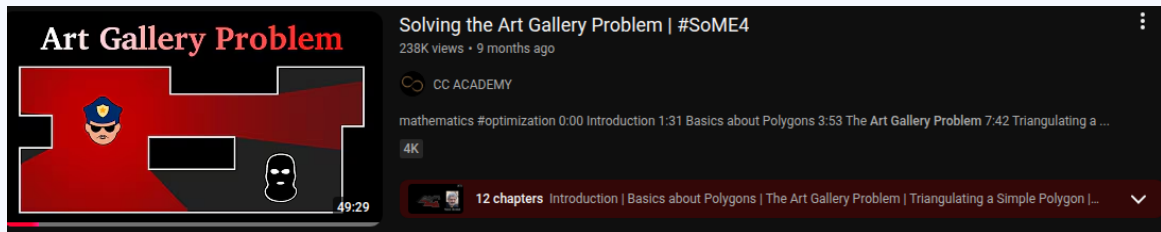
<https://github.com/abd-faiyaz/CSE-462-Art-Gallery-Problem>

Quick Reproduction

```
git clone https://github.com/abd-faiyaz/CSE-462-Art-Gallery-Problem.git && cd  
CSE-462-Art-Gallery-Problem && ./run_all.sh
```

Contains all code, datasets, and instructions required to reproduce the plots shown in this presentation.

Acknowledgements



Video

https://youtu.be/idqAkx_zP_c

References

- [1] Chvátal, V. (1975). A combinatorial theorem in plane geometry. *Journal of Combinatorial Theory, Series B*, 18:39–41.
- [2] Fisk, S. (1978). A short proof of Chvátal's watchman theorem. *Journal of Combinatorial Theory, Series B*, 24(3):374–375.
- [3] Lee, D.-T. & Lin, A. K. (1986). Computational complexity of art gallery problems. *IEEE Trans. on Information Theory*, 32(2):276–282.
- [4] O'Rourke, J. (1987). *Art Gallery Theorems and Algorithms*. Oxford University Press.
- [5] Abrahamsen, M., Adamaszek, A. & Miltzow, T. (2018). The art gallery problem is $\exists\mathbb{R}$ -complete. *STOC 2018*, pp. 65–73.
- [6] Eidenbenz, S., Stamm, C. & Widmayer, P. (2001). Inapproximability results for guarding polygons and terrains. *Algorithmica*, 31(1):79–113.
- [7] Amit, Y., Mitchell, J. S. B. & Packer, E. (2007). Locating guards for visibility coverage of polygons. *ALLENEX 2007*, pp. 120–134.
- [8] Ghosh, S. K. (2010). Approximation algorithms for art gallery problems in polygons. *Discrete Applied Mathematics*, 158(6):718–722.
- [9] Bonnet, É. & Miltzow, T. (2017). An approximation algorithm for the art gallery problem. *SoCG 2017, LIPIcs*, pp. 20:1–20:15.
- [10] Couto, M. C., de Rezende, P. J. & de Souza, C. C. (2011). An exact algorithm for minimizing vertex guards on art galleries. *Int. Trans. in Operational Research*, 18:425–448.
- [11] Kröller, A., Baumgartner, T., Fekete, S. P. & Schmidt, C. (2012). Exact solutions and bounds for general art gallery problems. *ACM J. Experimental Algorithmics*, 17(2.3).
- [12] Bottino, A. & Laurentini, A. (2011). A nearly optimal algorithm for covering the interior of an art gallery. *Pattern Recognition*, 44(5):1048–1056.
- [13] Tozoni, D. C., de Rezende, P. J. & de Souza, C. C. (2016). Algorithm 966: A practical iterative algorithm for the art gallery problem using ILP. *ACM Trans. Math. Software*, 43(2), Article 16.
- [14] Hengeveld, S. B. & Miltzow, T. (2021). A practical algorithm with performance guarantees for the art gallery problem. *SoCG 2021, LIPIcs Vol. 189, Article 44*, pp. 44:1–44:16.
- [15] de Rezende, P. J., de Souza, C. C., Friedrichs, S., Hemmer, M., Kröller, A. & Tozoni, D. C. (2016). Engineering art galleries. *Algorithm Engineering*, Springer, pp. 379–417.
- [16] Couto, M. C., de Rezende, P. J. & de Souza, C. C. (2009). Instances for the Art Gallery Problem. www.ic.unicamp.br/~cid/Problem-instances/Art-Gallery

Thank You

Questions & Discussion

CSE 462 — Algorithm Engineering Sessional
Department of CSE, BUET